GUIDE 62 · OPERATE

See what's being used, from the Console

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=62-see-whats-being-used-from-the-console. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a clear read on which keys, connections, queries and tools are actually in use - which ones are busy, which are granted but never touched - without wiring up any external monitoring.

There are two kinds of usage view in the Console, and it helps to know which you're looking at:

- **Activity over the last 30 days** - drawn from the run log, so it survives restarts. This is the sparklines and the per-key usage history below.
- **Live counts since this server process started** - in-memory, reset on restart. This is the "Usage" column on each table, the Metrics screen, and `/metrics`.

1. Activity sparklines: who's busy, who's idle

Five Console tables now carry a small 30-day **activity sparkline** per row - a GitHub-style graph of runs per day:

SCREEN	ONE SPARKLINE PER…	SHOWS
Access → API keys	key	how much each key is used (and links to its details)
Access → Roles	role	the combined activity of the keys created from that role
Data → Connections	connection	traffic to each database
Data → Caching	cached query	which `cache_ttl` queries actually get hit
MCP server	tool	a saved-query tool's **MCP** calls (built-in tools show "—")

A drawn line means traffic; a **flat baseline means "granted but never used in 30 days"** - often the most useful thing on the screen, because it points at access you can take away. Hover any sparkline for the totals (for example *"example-reporting: 28 runs over 30 days · 3 failed · peak 3/day"*).

2. A key in full: the Details screen

On **Access → API keys**, each row has a **Details** link. It opens a page built entirely from that key's own activity:

- the saved queries (APIs) it can reach, and how each is granted;
- the connections and tables behind them, its limits and masking;
- **last used**, and its full **run history** - every call, with when, what, rows, duration and result;
- a per-API sparkline, scoped to this key, so you can see which of its granted APIs it actually calls.

Clicking a query there opens its SQL. This is the fastest way to answer "what has this partner's key been doing?"

3. Who's using how much: usage quotas

Usage quotas (ADR 0011) count calls and rows per caller, per day and per month, in the store - so they're durable like the sparklines. `GET /api/v1/usage/quotas` is what the Console reads:

```shell
curl http://127.0.0.1:5000/api/v1/usage/quotas -H 'X-API-Key: demo-key'
```

```json
{"items": [
  {"caller": "example-dashboard", "per": "month", "period": "2026-10", "calls": 24, "rows": 8013, ...},
  {"caller": "example-executive", "per": "month", "period": "2026-10", "calls": 19, "rows": 6937, ...},
  ...]}
```

These are the numbers a key's quota is measured against (Give a partner a quota and a concurrency cap).

4. Server-wide, right now: the Metrics screen

Observability → Metrics shows request volume, latency and error rate across the whole server. These come from in-memory counters, the same ones `/metrics` exposes in Prometheus format - so they count **only since this process started**, and a restart resets them. That's why a freshly restarted server shows low "Usage" numbers on the key and connection tables even when the 30-day sparklines are full: the sparklines read the durable run log, the Usage column reads these live counters.

For dashboards and alerting over time, scrape `/metrics` - Wire up Prometheus and Grafana.

5. What agents did: the Agents screen

Observability → Agents is the usage view for AI clients: what each agent called over MCP, in the recent window. `GET /api/v1/usage/agents` backs it:

```shell
curl http://127.0.0.1:5000/api/v1/usage/agents -H 'X-API-Key: demo-key'
# {"since": "...", "until": "...", "sample_rate": 1.0, "truncated": false, "items": [...]}
```

For the data dictionary agents see, and more on this screen, see Tell agents what your data means, and see what they did.

6. The run log behind all of it

Every view above the sparklines and histories is the run log, read different ways. Query it directly, filtered by key, query, status, kind or a time window:

```shell
curl 'http://127.0.0.1:5000/api/v1/history?key=example-reporting&limit=3' -H 'X-API-Key: demo-key'
```

```json
{"items": [
  {"query": "example_top_films", "kind": "saved", "executed_at": "2026-10-19 05:51:56",
   "connection_name": "examples", "transport": "rest", "status": "success", "rows": 613, "duration_ms":
748},
  ...],
 "next_cursor": null}
```

For a *live* feed rather than history, the **Home** screen's requests panel updates in real time as queries run (over `/events`), and you can build your own - Build a client that watches queries run.

Next steps

- Wire up Prometheus and Grafana - the live counters as dashboards and alerts.
- Read the audit log - not *what ran*, but *who changed the configuration*.
- Send the audit log to your SIEM - ship the record off the box.